

- ✧ Tutorial 7: CN7050: Intelligent Systems

Dr Shaheen Khatoon

Ph.D. | PMP | PMI-ACP | ITIL | PgCert TLHE

Senior Lecturer in Data Science (Computer Science and Digital Technologies) School of Architecture, Computing and Engineering

Room: EB.1.99

(e) S.Khatoon@uel.ac.uk

(w) <https://uel.ac.uk/about-uel/staff/shaheen-khatoun>'''

Developing RAG Based Applications

This is a graded lab, worth 5% marks. You must complete the Lab task and submit via moodle link.

The lab task require you to complete the code given in the task at the end of this notebook.

In this tutorial, you will learn about:

Baisc RAG pipeline

Examples of Developing RAG using text and PDF as external sources

Your Tasks:

Towards the end you will follow the same steps and develop a RAG pipeline for using web pages as external data source

For this exercise, we will load external documents in text and PDF form, and develop a RAG Application

- Setting Up the Environment

First, ensure you have the necessary libraries installed.

These include `*langchain*` for the core framework, HuggingFace models, `pypdf` for loading PDF files, `Faiss /chromadb` for the vector store etc.

```
1 %%capture
2 !pip install langchain==0.2.5 faiss-cpu==1.8.0 langchain-community==0.2.5 sentence-transformers==
```

```
1 !pip install llama-cpp-python==0.2.78 --extra-index-url https://abetlen.github.io/llama-cpp-pyth
2
3 ## IMPORTANT: Make sure to restart the session after installing the packages above.
```

```
Looking in indexes: https://pypi.org/simple, https://abetlen.github.io/llama-cpp-python/whl/cu124
Collecting llama-cpp-python==0.2.78
  Downloading llama_cpp_python-0.2.78.tar.gz (50.2 MB)
    

---

50.2/50.2 MB 9.6 MB/s eta 0:00:00
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Installing backend dependencies ... done
  Preparing metadata (pyproject.toml) ... done
Requirement already satisfied: typing-extensions>=4.5.0 in /usr/local/lib/python3.12/dist-packages (from llama-cpp-python==0.2.78)
Requirement already satisfied: numpy>=1.20.0 in /usr/local/lib/python3.12/dist-packages (from llama-cpp-python==0.2.78) (1.26.4)
Collecting diskcache>=5.6.1 (from llama-cpp-python==0.2.78)
  Downloading diskcache-5.6.3-py3-none-any.whl.metadata (20 kB)
Requirement already satisfied: Jinja2>=2.11.3 in /usr/local/lib/python3.12/dist-packages (from llama-cpp-python==0.2.78) (3.1.4)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2>=2.11.3->llama-cpp-python==0.2.78) (3.0.2)
  Downloading diskcache-5.6.3-py3-none-any.whl (45 kB)
    

---

45.5/45.5 kB 3.6 MB/s eta 0:00:00
Building wheels for collected packages: llama-cpp-python
  Building wheel for llama-cpp-python (pyproject.toml) ... done
  Created wheel for llama-cpp-python: filename=llama_cpp_python-0.2.78-cp312-cp312-linux_x86_64.whl size=3742457 sha256=2d7f5e5e5e5e5e5e5e5e5e5e5e5e5e5e5e5e5e5e5e5e5e5e5e5e5e5e5e5e5e5e
  Stored in directory: /root/.cache/pip/wheels/ee/f8/a8/978da1e57d80f11bb70643e0e0d2852bdc14a5e25c42af28ea
Successfully built llama-cpp-python
Installing collected packages: diskcache, llama-cpp-python
```

Successfully installed diskcache-5.6.3 llama-cpp-python-0.2.78

- ✓ Example 1: RAG with Local Models On Text
- ✓ Preparing and Splitting the Document (Text)

```

1 text = """
2 Interstellar is a 2014 epic science fiction film co-written, directed, and produced by Christophe
3 It stars Matthew McConaughey, Anne Hathaway, Jessica Chastain, Bill Irwin, Ellen Burstyn, Matt Da
4 Set in a dystopian future where humanity is struggling to survive, the film follows a group of as
5
6 Brothers Christopher and Jonathan Nolan wrote the screenplay, which had its origins in a script J
7 Caltech theoretical physicist and 2017 Nobel laureate in Physics[4] Kip Thorne was an executive p
8 Cinematographer Hoyte van Hoytema shot it on 35 mm movie film in the Panavision anamorphic format
9 Principal photography began in late 2013 and took place in Alberta, Iceland, and Los Angeles.
10 Interstellar uses extensive practical and miniature effects and the company Double Negative creat
11
12 Interstellar premiered on October 26, 2014, in Los Angeles.
13 In the United States, it was first released on film stock, expanding to venues using digital proj
14 The film had a worldwide gross over $677 million (and $773 million with subsequent re-releases),
15 It received acclaim for its performances, direction, screenplay, musical score, visual effects, a
16 It has also received praise from many astronomers for its scientific accuracy and portrayal of th
17 Interstellar was nominated for five awards at the 87th Academy Awards, winning Best Visual Effect
18

```

```

1 # Split into a list of sentences
2 texts = text.split('.')
3
4 # Clean up to remove empty spaces and new lines
5 texts = [t.strip(' \n') for t in texts]

```

- ✓ Loading the Generation Model

```
1 !wget https://huggingface.co/microsoft/Phi-3-mini-4k-instruct-gguf/resolve/main/Phi-3-mini-4k-ins
```

```

--2025-11-06 13:04:02-- https://huggingface.co/microsoft/Phi-3-mini-4k-instruct-gguf/resolve/main/Phi-3-mini-4k-instruct-q4
Resolving huggingface.co (huggingface.co)... 3.168.132.126, 3.168.132.38, 3.168.132.91, ...
Connecting to huggingface.co (huggingface.co)|3.168.132.126|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://cas-bridge.xethub.hf.co/xet-bridge-us/662698108f7573e6a6478546/df220524a4e4a750fe1c325e41f09ff69137f38b52c
--2025-11-06 13:04:02-- https://cas-bridge.xethub.hf.co/xet-bridge-us/662698108f7573e6a6478546/df220524a4e4a750fe1c325e41f0
Resolving cas-bridge.xethub.hf.co (cas-bridge.xethub.hf.co)... 18.164.174.68, 18.164.174.4, 18.164.174.110, ...
Connecting to cas-bridge.xethub.hf.co (cas-bridge.xethub.hf.co)|18.164.174.68|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 2393231072 (2.2G)
Saving to: 'Phi-3-mini-4k-instruct-q4.gguf'

Phi-3-mini-4k-instr 100%[=====] 2.23G 110MB/s in 13s

2025-11-06 13:04:16 (171 MB/s) - 'Phi-3-mini-4k-instruct-q4.gguf' saved [2393231072/2393231072]

```

```
1 from langchain import LlamaCpp
2
3 # Make sure the model path is correct for your system!
4 llm = LlamaCpp(
5     model_path="Phi-3-mini-4k-instruct-q4.gguf",
6     n_gpu_layers=-1,
7     max_tokens=500,
8     n_ctx=2048,
9     seed=42,
10    verbose=False
11 )
```

✓ Loading the Embedding Model

An embedding is a dense vector, which is essentially a list of numbers, that represents a piece of text in a multi-dimensional space. These vectors are generated by specialized models trained to capture the semantic properties of language. The defining characteristic of a well-constructed embedding space is that texts with similar meanings will have vectors that are close to one another.

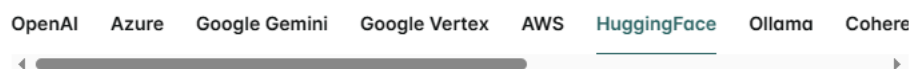
LangChain provides a standard interface for many embedding model providers. To generate an embedding, you instantiate a class for your chosen provider, such as **OpenAIEmbeddings** or **HuggingFaceEmbeddings**, and use its `embed_query` method. List of embedding available here:

<https://docs.langchain.com/oss/python/langchain/knowledge-base#huggingface>

2. Embeddings

Vector search is a common way to store and search over unstructured data (such as unstructured text). The idea is to store numeric vectors that are associated with the text. Given a query, we can **embed** it as a vector of the same dimension and use vector similarity metrics (such as cosine similarity) to identify related text.

LangChain supports embeddings from dozens of providers. These models specify how text should be converted into a numeric vector. Let's select a model:



we will choose the BAAI/bge-small-en-v1.5 model. it is relatively small:

```
1 from langchain.embeddings.huggingface import HuggingFaceEmbeddings
2
3 # Embedding Model for converting text to numerical representations
4 embedding_model = HuggingFaceEmbeddings(
5     model_name='BAAI/bge-small-en-v1.5'
6 )
```

```

/tmp/ipython-input-1728599694.py:4: LangChainDeprecationWarning: The class `HuggingFaceEmbeddings` was deprecated in LangChain
embedding_model = HuggingFaceEmbeddings(
/usr/local/lib/python3.12/dist-packages/sentence_transformers/cross_encoder/CrossEncoder.py:11: TqdmExperimentalWarning: Usi
from tqdm.autonotebook import tqdm, trange
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
modules.json: 100% 349/349 [00:00<00:00, 16.6kB/s]

config_sentence_transformers.json: 100% 124/124 [00:00<00:00, 13.6kB/s]

README.md: 94.8k/? [00:00<00:00, 8.75MB/s]

sentence_bert_config.json: 100% 52.0/52.0 [00:00<00:00, 5.66kB/s]

config.json: 100% 743/743 [00:00<00:00, 79.4kB/s]

model.safetensors: 100% 133M/133M [00:01<00:00, 155MB/s]

tokenizer_config.json: 100% 366/366 [00:00<00:00, 39.9kB/s]

vocab.txt: 232k/? [00:00<00:00, 10.1MB/s]

tokenizer.json: 711k/? [00:00<00:00, 32.3MB/s]

special_tokens_map.json: 100% 125/125 [00:00<00:00, 12.8kB/s]

config.json: 100% 190/190 [00:00<00:00, 20.7kB/s]

```

✓ Preparing the Vector Database

LangChain integrates with dozens of vector stores, from lightweight, in-memory libraries like: ** Chroma and FAISS to production-grade, standalone databases like Pinecone and Weaviate. ** see a complete list here:

<https://docs.langchain.com/oss/python/langchain/knowledge-base#chroma>

we use FAISS in this example, however, For development, an in-memory option like Chroma is an excellent starting point.

3. Vector stores

LangChain **VectorStore** objects contain methods for adding text and **Document** objects to the store, and querying them using various similarity metrics. They are often initialized with **embedding** models, which determine how text data is translated to numeric vectors.

LangChain includes a suite of **integrations** with different vector store technologies. Some vector stores are hosted by a provider (e.g., various cloud providers) and require specific credentials to use; some (such as **Postgres**) run in separate infrastructure that can be run locally or via a third-party; others can run in-memory for lightweight workloads. Let's select a vector store:

In-memory AstraDB Chroma **FAISS** Milvus MongoDB PGVector PGVectorStore

We can now use the embedding model to set up our vector database:

```

1 from langchain.vectorstores import FAISS
2
3 # Create a local vector database
4 db = FAISS.from_texts(texts, embedding_model)

```

✓ Developing RAG Prompt

A prompt template plays a vital part in the RAG pipeline.

It is the central place where we communicate the relevant documents to the LLM. To do so, we will create an additional input variable named **context** that can provide the LLM with the retrieved documents:

```
1 from langchain import PromptTemplate
2 from langchain.chains import RetrievalQA
```

```
1 # Create a prompt template
2 template = """<|user|>
3 Relevant information:
4 {context}
5
6 Provide a concise answer the following question using the relevant information provided above:
7 {question}<|end|>
8 <|assistant|>"""
9
10 prompt = PromptTemplate(
11     template=template,
12     input_variables=["context", "question"]
13 )
```

✓ Building a Question-Answering Chain

Building a Retrieval Augmented Generation system utilizes several essential components. These include a retriever designed to fetch relevant documents from a vector store and an LLM capable of generating text. Combining these elements creates a cohesive application, allowing it to take a user's question and produce a well-grounded answer.

Creating a RetrievalQA chain is simple. The most common constructor, `from_chain_type`, requires three main arguments:

llm: The language model that will generate the final answer.

chain_type: A string that specifies how the retrieved documents will be combined and passed to the LLM. We will start with "stuff".

****retriever:**** The configured retriever object responsible for fetching documents. A retriever is an object that implements a common interface for fetching documents based on a query. Its primary function is to accept a string query and return a list of Document objects.

```
1 Start coding or generate with AI.
```

```
1 # RAG Pipeline
2 rag = RetrievalQA.from_chain_type(
3     llm=llm,
4     chain_type='stuff',
5     retriever=db.as_retriever(),
6     chain_type_kwargs={
7         "prompt": prompt
8     },
9     verbose=True
10 )
```

Now we're ready to call the model and ask it a question:

```
1 rag.invoke('Income generated')
```

```
> Entering new RetrievalQA chain...
```

```
> Finished chain.
```

```
{'query': 'Income generated',
 'result': ' The provided information does not directly relate to income generation. However, considering the success of "Interstellar" as mentioned (the involvement of Kip Thorne and its release methods), it can be inferred that "Interstellar" was a commercially successful film which likely contributed positively to income generated through box office sales and ancillary revenue streams such as home media, streaming rights, and the tie-in book. But specific financial figures are not given in the information provided.'}
```

As always, we can adjust the prompt to control the model's generation (e.g., answer length and tone). as shown at the end of example 2

✓ Example 2: RAG Pipeline Over PDF Documents

in the below example we will be using the same steps, however, the datasource will be used is pdf document.

All steps are almost the same, only you need to use necessary libraries to process pdf documents

✓ Loading PDF Documents

```
1 !pip install pypdf
```

```
Collecting pypdf
  Downloading pypdf-6.1.3-py3-none-any.whl.metadata (7.1 kB)
  Downloading pypdf-6.1.3-py3-none-any.whl (323 kB)
    _____ 323.9/323.9 kB 6.5 MB/s eta 0:00:00
Installing collected packages: pypdf
Successfully installed pypdf-6.1.3
```

```
1 # You may need to install the pypdf library: pip install pypdf
2 from langchain_community.document_loaders import PyPDFLoader
3
4 loader = PyPDFLoader("CN7050 Coursework 25-26.pdf") #use any pdf document
5 pdfdoc = loader.load()
6
7 # Assuming the PDF has multiple pages
8 print(f"Loaded {len(pdfdoc)} pages from the PDF.")
9
10 # Inspect the first page's document
11 first_page_doc = pdfdoc[0]
12 print(first_page_doc.page_content[:200]) # Print first 200 characters
13 # ... content of the first page ...
14
15 print(first_page_doc.metadata)
16 # {'source': 'product_manual.pdf', 'page': 0}
```

```
Loaded 5 pages from the PDF.
```

```
1
  SCHOOL OF ARCHITECTURE, COMPUTING, & ENGINEERING CN7050 Coursework Guidelines Coursework instructions ☐ Include a Cover p
{'source': 'CN7050 Coursework 25-26.pdf', 'page': 0}
```

✓ Splitting Documents for Processing

When working with LLM applications, documents are often represented as Document objects, which can contain a substantial amount of text. Directly feeding a large document to an LLM for question-answering is impractical for two primary reasons. First, it will likely exceed the model's context window limit. Second, providing a large, unfocused block of text makes it difficult for the model to locate the specific information needed to answer a query.

The solution is to break down large documents into smaller, semantically meaningful chunks. This process, known as **text splitting or chunking**, is a significant step in preparing data for a RAG system.

The goal is to create text fragments that are small enough to be efficiently processed by an embedding model yet large enough to retain their original context.

✓ The Chunking Trade-off: Size vs. Context

Choosing the right chunk size involves a delicate balance. If chunks are too small, the semantic context can be lost.

For example, a chunk containing only the phrase "it was a major breakthrough"

is meaningless without the preceding sentences that explain what "it" refers to. On the other hand, if chunks are too large, they may contain irrelevant information that creates noise during retrieval, making it harder for the vector search to identify the most relevant passage.

This is often called the "lost in the middle" problem, where important information is buried within a large, noisy text block.

✓ Recursive Character Splitting

The most recommended and versatile method for text splitting in LangChain is the RecursiveCharacterTextSplitter. Instead of splitting on a fixed character, this splitter works with a prioritized list of separators. It attempts to split the text using the first separator in the list (by default, ["\n\n", "\n", " ", ""]). If the resulting chunks are still too large, it moves to the next separator, and so on.

This hierarchical approach naturally aligns with the structure of written text. It tries to keep paragraphs together first, then sentences, and finally words, ensuring the resulting chunks are as semantically coherent as possible.

Two important parameters control its behavior:

chunk_size: Defines the maximum length of each chunk, measured in characters.

chunk_overlap: Specifies the number of characters to overlap between adjacent chunks. This is a powerful feature that helps maintain continuity of context. If a sentence is cut off at the end of one chunk, the overlap ensures it is completed in the next, reducing the chance of losing relationships between sentences.

```
1 from langchain_text_splitters import RecursiveCharacterTextSplitter
2
3 text_splitter = RecursiveCharacterTextSplitter(chunk_size = 1000, chunk_overlap
  = 200)
4 splits = text_splitter.split_documents(pdfdoc)
```

```
1 print(splits[0])
2 print(splits[1])
3 print(splits[2])
```

```
page_content='1' metadata={'source': 'CN7050 Coursework 25-26.pdf', 'page': 0}
page_content='SCHOOL OF ARCHITECTURE, COMPUTING, & ENGINEERING CN7050 Coursework Guidelines Coursework instructions & Inclu
page_content='Learning Outcomes: 1-8 Turnitin submission requirement Yes Additional information ASSESSMENT FEEDBACK -
```

```
1 print(len(splits))
```

```
14
```

✓ Create Vector Database

```
1 # Create a local vector database
2 # Extract the text content from each Document object
3 texts_from_pdf = [doc.page_content for doc in splits]
4 PDFdb = FAISS.from_texts(texts_from_pdf, embedding_model)
```

✓ Build Prompt and QA Chain

```
1 from langchain import PromptTemplate
2 from langchain.chains import RetrievalQA
3
4
5 # Create a prompt template
6 template = """<|user|>
7 Relevant information:
8 {context}
9
10 Provide a concise answer the following question using the relevant information provided above:
11 {question}<|end|>
12 <|assistant|>"""
13
14 prompt = PromptTemplate(
15     template=template,
16     input_variables=["context", "question"]
17 )
18
19 # RAG Pipeline
20 rag = RetrievalQA.from_chain_type(
21     llm=llm,
22     chain_type='stuff',
23     retriever=PDFdb.as_retriever(),
24     chain_type_kwargs={
25         "prompt": prompt
26     },
27     verbose=True
28 )
```

Running this code will take long time as we are using Open source models.

```
1 rag.invoke('submission date')
```

> Entering new RetrievalQA chain...

> Finished chain.

```
{'query': 'submission date',
 'result': ' Submission Date: Friday, December 19th , 2025, 11:55PM\n\nTask 1 Report - Intelligent Systems - Skills Development (Your Id-Name)\n\nModule Code: CN7050 \n\nModule Title: Intelligent Systems \n\nModule Leader: Dr. Azhar Mahmood \n\nAssignment Tutor: Lectures by Shaheen Khatoon, Azhar Mahmood; Lab Instructors by Dr. Shaheen Khatoon, Dr. Mustansar Ali Ghanzafar, Dr. Shahera Hossain, Zulfiqar Ali, Afroza Rahman, Anas Mahdaoui, Fiza Riaz, Maimoona Sharif\n\nAssignment Title: Intelligent Systems Coursework Assignment 1/1 \n\nWeighting Individual Assignment: 100% \n\nHandout Date (W3): [Specific date provided in module] \n\nSubmission Date: Friday, December 19th , 2025, 11:55PM \n\nLearning Outcomes (1-8) Assessed by this assignment: \n\n- Explain how the system is initiated or triggered. \n- Describe any preprocessing, encoding, or transformation applied to data before running simulations. \n- Agent Training and Testing Process \n- Explain how agents are trained, tested, and validated. \n- Include details such as data distribution ratios (training/testing/validation) and evaluation metrics.\n\nTurnitin Submission Link: [Provide the link from the module's Moodle site]'}
```

✓ Refining Results:

Model will not give direct answer if prompt is not appropriately instructing the model how the results should look like: there are ways to get desired results for example

Update Prompt Template: provide as clear instruction to the model as possible

format results according to required format

```
1 #update prompt to see if different answer will be generated
2 from langchain import PromptTemplate
3
4 template = """<|user|>
5 You are an assistant that extracts specific information from academic PDF documents.
6
7 Context from document:
8 {context}
9
10 Question: {question}
11
12 Instruction:
13 - Provide only the direct answer.
14 - Do not include explanations, context, or repeated information.
15 - If the answer is a date, return only the date in this format: YYYY-MM-DD or the exact format fo
16 <|end|>
17 <|assistant|>"""
18
19 prompt = PromptTemplate(
20     template=template,
21     input_variables=["context", "question"]
22 )
23
```

```
1 prompt = PromptTemplate(
2     template=template,
3     input_variables=["context", "question"]
4 )
```

```
1 response = rag.invoke("What is the submission date of the assignment?")
2 print(response["result"])
```

> Entering new RetrievalQA chain...

> Finished chain.

The submission date of the assignment, as mentioned in the coursework guidelines for Task 2 (Design, Implement and Report a

✓ Lab Tasks:

Document loaders provide a standard interface for reading data from different sources. see the different sources from here:

https://docs.langchain.com/oss/python/integrations/document_loaders/index#webpages

After practicing with the above two example, select a different document loader for in this case WebbasedLoader to develop a RAG pipeline for web pages.

To load content directly from a URL, you can use the **WebBaseLoader**.

It fetches the HTML from the given URL, parses it, and extracts the text content into a Document.

This is an efficient way to pull in online articles, blog posts, or documentation.

Initial code for loading an example web page is provided below: apply the remaining steps such as document splitting, creating vector Db, defining prompt and building and running RAG pipeline.

use help from here to get better understanding:

<https://docs.langchain.com/oss/python/langchain/rag#google-gemini>

✓ Load Webpages

```
1 !pip install beautifulsoup4
```

```
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.12/dist-packages (4.13.5)
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.12/dist-packages (from beautifulsoup4) (2.8)
Requirement already satisfied: typing-extensions>=4.0.0 in /usr/local/lib/python3.12/dist-packages (from beautifulsoup4) (4.12.0)
```

```
1 # You may need to install BeautifulSoup:
2 # pip install beautifulsoup4
3 import bs4
4 from langchain_community.document_loaders import WebBaseLoader
5
6 # Only keep post title, headers, and content from the full HTML.
7
8 # loader = WebBaseLoader(web_paths=["https://lilianweng.github.io/posts/2023-06-23-agent/"]) #chan
9
10 bs4_strainer = bs4.SoupStrainer(class_=("post-title", "post-header", "post-content"))
11 loader = WebBaseLoader(
12     web_paths=("https://lilianweng.github.io/posts/2023-06-23-agent/",),
13     bs_kwargs={"parse_only": bs4_strainer},
14 )
```

```
1 Webdocs = loader.load()
```

```
1 assert len(Webdocs) == 1
2 print(f"Total characters: {len(Webdocs[0].page_content)}")
```

```
Total characters: 43047
```

```
1 print(Webdocs[0].page_content[:500])
2 # ... a clean text version of the webpage's content ...
3
4 print(Webdocs[0].metadata)
```

LLM Powered Autonomous Agents

Date: June 23, 2023 | Estimated Reading Time: 31 min | Author: Lilian Weng

Building agents with LLM (large language model) as its core controller is a cool concept. Several proof-of-concepts demos, s
Agent System Overview#
In
{'source': 'https://lilianweng.github.io/posts/2023-06-23-agent/'}

✓ Split Documents

use the below text splitters

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
```

```
1 # your code should go here
```

▼ Create Vector Database

Note: if you are running in this notebook, you dont need to load embedding model and LLM again

however, if you are completing the task in a new notebook, you must load embedding and text generation model before creating Vector database.

```
1 # you cde should go here
```

▼ Create Prompt and QA Chain

```
1 # Your code
```

```
1 # you code to run rag
```

Submission Instructions

After completing the tasks, follow the steps below:

Copy your code along with the corresponding results (if any) for each section of the tasks into a Word document.

You may include screenshots of the results where appropriate.

Explain your results clearly to demonstrate your understanding of the models/results.

Provide additional explanations or interpretations wherever required.

Convert the completed document to a PDF file.